



MODUL 14 · SEO, METADATA & PERFORMANCE

sitemap.ts & robots.ts



Static dan dynamic metadata. Open Sitemap dan robots.txt yang generate otomatis dari data DB — native Next.js, tanpa library.Graph, Twitter Cards, icons. Semua native di Next.js.

SEO & PERFORMANCE

sitemap.ts — Sitemap Otomatis dari Database

Buat app/sitemap.ts yang return array URL. Next.js generate /sitemap.xml otomatis.

```
import type { MetadataRoute } from "next"
import { db } from "@lib/db"

export const revalidate = 3600 // Regenerate sitemap setiap 1 jam

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const baseUrl = process.env.NEXT_PUBLIC_APP_URL!

  // Static pages
  const staticPages: MetadataRoute.Sitemap = [
    { url: baseUrl, lastModified: new Date(), changeFrequency: "daily", priority: 1 },
    { url: `${baseUrl}/blog`, lastModified: new Date(), changeFrequency: "hourly", priority: 0.9 },
    { url: `${baseUrl}/about`, lastModified: new Date(), changeFrequency: "monthly", priority: 0.5 }
  ],

  // Dynamic pages dari DB — semua published posts
  const posts = await db.post.findMany({
    where: { published: true, deletedAt: null },
    select: { slug: true, updatedAt: true },
    orderBy: { updatedAt: "desc" },
  })

  const postPages: MetadataRoute.Sitemap = posts.map(post => ({
    url: `${baseUrl}/blog/${post.slug}`,
    lastModified: post.updatedAt,
    changeFrequency: "weekly",
    priority: 0.8,
  }))

  return [ ...staticPages, ...postPages ]
}

// Hasil di /sitemap.xml:
// <?xml version="1.0" encoding="UTF-8"?>
// <urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
//   <url><loc>https://site.com</loc><priority>1</priority></url>
//   <url><loc>https://site.com/blog/my-post</loc> ... </url>
// </urlset>
```

robots.ts — Kontrol Crawler

Tentukan halaman mana yang boleh dan tidak boleh di-crawl oleh search engine bot.

```
import type { MetadataRoute } from "next"

export default function robots(): MetadataRoute.Robots {
  const baseUrl = process.env.NEXT_PUBLIC_APP_URL!

  return {
    rules: [
      {
        userAgent: "*", // semua crawler
        allow: ["/"], // boleh crawl semua
        disallow: [
          "/api/", // API endpoints tidak
          "/admin/", // halaman admin private
          "/dashboard/", // dashboard user private
          "/profile/settings/", // settings private
        ],
      },
      // Specific crawler rules (optional):
      {
        userAgent: "GPTBot", // ChatGPT crawler
        disallow: ["/"], // blokir AI training
      },
    ],
    sitemap: `${baseUrl}/sitemap.xml`,
    // Tambahkan URL sitemap ke robots.txt
  }
}

// Hasil di /robots.txt:
// User-agent: *
// Allow: /
// Disallow: /api/
// Disallow: /admin/
// Sitemap: https://site.com/sitemap.xml
```

Apa yang perlu di-disallow?

✅ Disallow (tidak perlu crawl):
/api/* — response JSON
/admin/* — akses terbatas
/dashboard/* — konten personal
/_next/* — Next.js internal
/private/* — konten private

✅ Allow (perlu crawl):
/ — homepage
/blog/* — konten publik
/about — about page
/products/* — konten publik

GPTBot (ChatGPT):
Jika tidak mau konten dipakai
untuk training AI, tambahkan
rule khusus untuk GPTBot.

JSON-LD — Structured Data untuk Rich Results

Structured data memungkinkan rich snippets di Google: breadcrumbs, article metadata, rating.

```
export default async function PostPage({ params }) {
  const { slug } = await params
  const post = await getPost(slug)
  if (!post) notFound()

  // JSON-LD Structured Data — inject ke <head>
  const jsonLd = {
    '@context': 'https://schema.org',
    '@type': 'BlogPosting',
    headline: post.title,
    description: post.excerpt,
    datePublished: post.createdAt.toISOString(),
    dateModified: post.updatedAt.toISOString(),
    author: {
      '@type': 'Person',
      name: post.author.name,
    },
    image: post.coverImage,
    publisher: {
      '@type': 'Organization',
      name: 'EasyCoding',
    },
    mainEntityOfPage: {
      '@type': '@id',
      '@id': `${process.env.NEXT_PUBLIC_APP_URL}/blog/${slug}`,
    },
  }

  return (
    <
      < /* JSON-LD: inject script tag di head */
      <script
        type="application/ld+json"
        dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd)
      </script>
      <article>
        <h1>{post.title}</h1>
        { /* ... */ }
      </article>
    </
  )
}
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Di mana file sitemap.ts diletakkan dan URL apa yang dihasilkan?

A) Di /public/sitemap.ts → /sitemap.txt

B) Di /app/sitemap.ts → /sitemap.xml otomatis

C) Di /pages/sitemap.ts → /sitemap.json

Q2 — Apa fungsi export const revalidate = 3600 di sitemap.ts?

A) Regenerate sitemap setiap 3600 detik (1 jam)

B) Cache sitemap selama 3600 menit

C) Limit 3600 URL per sitemap

Q3 — Mengapa /api/* sebaiknya di-disallow di robots.ts?

A) API endpoints tidak aman untuk crawler

B) JSON response tidak berguna untuk SEO dan hanya menambah beban server

C) Next.js tidak support crawl API routes

Tugas Mandiri

Implementasikan sitemap.ts dan robots.ts untuk project.

01 `app/sitemap.ts`

Buat sitemap dengan static pages (/, /blog, /about) + dynamic pages dari `db.post.findMany`. Set `revalidate = 3600`. Test di `/sitemap.xml`.

02 `app/robots.ts`

Buat robots.ts dengan rules: allow '/', disallow '/api/', '/admin/', '/dashboard/'. Tambahkan sitemap URL. Test di `/robots.txt`.

03 JSON-LD Post

Tambahkan JSON-LD BlogPosting schema ke `app/blog/[slug]/page.tsx`. Test di `search.google.com/test/rich-results`.

04 Sitemap Update

Di `createPost` Server Action: setelah publish post, tambahkan `revalidatePath('/sitemap.xml')`. Test: publish post baru → cek sitemap diupdate.



Test sitemap: `/sitemap.xml` di browser. Test robots: `/robots.txt`. Semua di-serve otomatis oleh Next.js.

Yang Sudah Kita Pelajari

- ✓ `app/sitemap.ts` → `/sitemap.xml` otomatis. Include static + dynamic pages dari DB.
- ✓ `revalidate = 3600`: regenerate sitemap tiap jam. `revalidatePath('/sitemap.xml')` on-demand.
- ✓ `app/robots.ts` → `/robots.txt`. Disallow `/api/`, `/admin/`, `/dashboard/`.
- ✓ JSON-LD structured data: inject `<script type='application/ld+json'>` untuk rich snippets.
- ✓ Test sitemap dan robots: langsung akses URL di browser. JSON-LD: rich-results test Google.

→ Selanjutnya: **Bab 3 — Font Optimization dengan next/font**

